

HACKATHON BATTLE PLAN

Agentic Edge powered by NemoClaw

Deep Tech Track • NVIDIA GPU Prize

System Design · Project Idea · Implementation Plan · Setup · Optimization · Demo Strategy

Stack: NemoClaw + NeMo Agent Toolkit + vLLM + Nemotron + Cursor

Built for a 24-hour hackathon. Every section is action-oriented.

Table of Contents

1. System Design: NemoClaw + vLLM Architecture
2. Project Idea: "AgentBench Live"
3. Implementation Plan (Hour-by-Hour)
4. Exact Setup Instructions
5. Optimization Tricks
6. Demo Strategy
- . Quick Reference Links

1. System Design: NemoClaw + vLLM Architecture

The Full Stack

The architecture has four layers. From top to bottom: your agent application code, the NeMo Agent Toolkit (NAT) middleware, the NemoClaw runtime sandbox, and the vLLM inference server running on GPU hardware. Each layer adds a distinct capability.

NeMo Agent Toolkit (NAT) — The Middleware

NAT sits between your agent code and the LLM endpoint. It does three things:

- **Tracing & Telemetry:** Instruments every LLM call, tool invocation, and memory access with start/stop events. The telemetry stream is **bidirectional** — it observes AND modifies agent behavior at runtime.
- **Agentic Headers (nnext):** Intercepts outgoing LLM requests and appends headers encoding predicted output sequence length (OSL), latency sensitivity scores, and request priority. vLLM's scheduler uses these for smarter batching and scheduling.
- **OSL Prediction via Trie Model:** Profiles 20% of your agent's tool-calling trajectories, builds a trie model, and predicts future OSL with ~90% accuracy ($\pm 10\%$ error). The inference server **knows how long the response will be before generating it**.

NemoClaw — The Runtime Sandbox

NemoClaw wraps everything in a sandboxed OpenShell environment with YAML-defined security policies. It enforces network isolation, file access controls, and data privacy. A privacy router splits requests between local Nemotron models and cloud endpoints. For the hackathon, it proves your agent operates within defined trust boundaries.

vLLM — The Inference Engine

Key features for agentic workloads:

- **Prefix Caching:** Agent loops hit the same system prompt + tool definitions repeatedly. vLLM caches KV blocks for shared prefixes — subsequent calls only compute new tokens. The meetup data showed **10x TTFT reduction** with KV cache reuse.
- **Continuous Batching:** New requests join running batches mid-generation. Parallel tool-call LLM invocations share GPU compute.
- **Speculative Decoding:** A draft model (EAGLE-3) proposes tokens verified in parallel. For structured JSON tool calls, acceptance rates are high → **1.7x decode speedup**.
- **Async Scheduling (Model Runner V2):** Overlaps CPU scheduling with GPU execution. Zero GPU stalls. MRV2 showed **56% throughput increase** on stress tests.
- **FP8 KV Cache:** Halves cache memory → doubles effective context length → 50% fewer GPUs needed.

Where Latency Bottlenecks Occur in Agent Loops

An agent loop is: User Query → LLM Call 1 (Plan) → Tool Calls → LLM Call 2 (Synthesize) → More Tools → LLM Call 3 (Final Answer). Bottlenecks in order of severity:

Rank	Bottleneck	Impact	Fix
1	Sequential LLM calls	Each call pays TTFT + gen time × N calls	Prefix caching, parallel calls

Rank	Bottleneck	Impact	Fix
2	Prefix re-computation	200-800ms wasted per call re-processing system prompt	Prefix caching (biggest single win)
3	Sequential tool execution	Tools block each other unnecessarily	Async parallel tool execution
4	Slow token generation	Autoregressive bottleneck on structured outputs	Speculative decoding
5	CPU scheduling overhead	GPU stalls between batches	Async scheduling (MRV2)

What 'Agentic Loop Optimization' Means in Practice

It means attacking all five bottlenecks simultaneously: reduce TTFT via prefix caching + FP8, reduce generation time via speculative decoding, parallelize independent tool calls, use NAT's agentic headers for priority-based scheduling, and predict+pre-allocate via OSL estimation. The meetup results showed these combined: **-89% Mean TTFT** under high utilization and **6.4x throughput** (535 → 3443 tok/s) going from round-robin to Thompson sampling with agentic hints.

2. Project Idea: "AgentBench Live"

A real-time agent performance profiler and auto-optimizer. It runs a complex multi-step agent task, profiles every LLM call and tool invocation, visualizes the critical path, automatically applies vLLM optimizations, re-runs the task, and shows the before/after delta.

Problem Statement

There is no standard way to **see** where an agentic workflow is slow, **why** it's slow, and **automatically fix it**. Existing frameworks give you logs. What you need is a system that profiles agent traces, identifies the critical path, and auto-tunes inference configuration to minimize end-to-end latency.

Why It's Novel

- Most hackathon agent projects are 'chatbot + tools.' This is a **systems-level performance tool**.
- Directly demonstrates NAT's telemetry and agentic headers in a visible, measurable way.
- Shows judges the **internals** of what makes agents fast — this is what 'Deep Tech' means.
- The optimization loop is itself agentic — an LLM analyzes its own traces and recommends config changes.

System Architecture

The system has five components arranged in layers:

- **AgentBench Live UI:** React dashboard with task picker, live Gantt chart of execution trace, before/after metrics panel, and LLM-generated optimization recommendations.
- **Benchmark Runner:** Runs the "Subject Agent" (a multi-step research agent with 4 LLM calls + 4-8 tool calls per task) in baseline mode (no optimizations) and optimized mode, collecting traces.
- **Optimizer Agent:** Ingests telemetry traces, identifies the critical path via dependency-graph analysis, detects parallelism opportunities, predicts OSL patterns, and generates vLLM configuration diffs.
- **NeMo Agent Toolkit Middleware:** Collects telemetry, builds OSL trie model during profiling, appends agentic headers to LLM requests.
- **vLLM Server:** Runs Nemotron-Nano (or Qwen3). Optimization modes toggled between runs: baseline → +prefix cache → +spec decode → +agentic headers.

Agent Loop Design

Two agents run in the system:

Agent 1: The 'Subject' Agent (benchmarked)

Given a complex question, it executes:

- LLM Call 1 — Decompose question into sub-queries (planning)
- Tool Calls — web_search + database_lookup in parallel
- LLM Call 2 — Synthesize partial results (intermediate reasoning)
- LLM Call 3 — Identify gaps, run additional searches
- LLM Call 4 — Produce structured final answer with citations

This is 4 LLM calls + 4-8 tool calls per task — enough to show meaningful optimization.

Agent 2: The 'Optimizer' Agent (meta-agent)

After baseline run: ingests telemetry, identifies critical path, calculates parallelism potential, predicts OSL patterns, generates vLLM config diff, and outputs human-readable recommendations like "Enable prefix caching: saves ~340ms/call × 3 = ~1.0s."

Where vLLM Is Leveraged

Optimization	What It Does	Expected Gain
Prefix Caching	Caches KV for shared system prompt + tool defs across all 4 LLM calls	3-10x TTFT reduction on calls 2-4
Speculative Decoding	Draft model proposes tokens for structured JSON tool-call outputs	1.5-1.7x decode speedup
FP8 Quantization	Reduces model memory + compute	50% less GPU, ~2x throughput
Async Scheduling	Overlaps CPU scheduling with GPU execution	Eliminates 5-15% idle time
Agentic Headers	Tells vLLM which requests are high-priority (first/last in chain)	-62% to -89% TTFT under load
Continuous Batching	Parallel tool-call LLM invocations share GPU batch	Better GPU utilization

3. Implementation Plan (Hour-by-Hour)

MVP by Hour 10: Subject Agent on vLLM with timing instrumentation, showing prefix caching ON vs OFF comparison in terminal. Everything else is additive polish.

Phase 1: Foundation (Hours 0–4)

Hour 0–1: Environment Setup

- Install vLLM on GPU machine, download and serve model (see Section 4)
- Verify OpenAI-compatible API works with tool calling
- Install NemoClaw + NeMo Agent Toolkit

Hour 1–2: Basic Agent Loop

- Build Subject Agent (Python, LangGraph or raw OpenAI client)
- Implement 3-4 tools: web_search (mock or real), calculator, database_query, summarizer
- Get single end-to-end run working: question → structured answer
- Log all timings manually (start/end per LLM call and tool)

Hour 2–3: Telemetry Integration

- Integrate NAT telemetry markers into agent loop
- Capture: LLM call durations, tool durations, token counts, TTFT per call
- Store traces as JSON for UI consumption

Hour 3–4: vLLM Configuration Variants

- Build config manager to restart/reconfigure vLLM with different flags
- Test prefix caching ON vs OFF, speculative decoding ON vs OFF
- Verify optimizations are toggleable between benchmark runs

Phase 2: Core System (Hours 4–10)

Hour 4–6: Benchmark Runner

- Build runner: task → run with config → collect traces → output JSON with baseline vs optimized metrics
- Run same task 3x per mode, average results for statistical validity

Hour 6–8: Optimizer Agent

- Agent 2 takes telemetry trace, outputs optimization recommendations
- Critical path analysis: dependency graph → longest path identification
- Parallelism detection: identify tool calls with no data dependencies
- Output: list of recommendations with predicted savings per optimization

Hour 8–10: Agentic Headers Integration

- Implement NAT middleware intercepting LLM requests
- Add nvext headers: predicted OSL, latency sensitivity, priority

- Measure impact of headers on TTFT under simulated load

Phase 3: UI + Demo Polish (Hours 10–18)

Hour 10–13: Live Dashboard

- React/HTML web UI: task picker, real-time Gantt chart (WebSocket), metrics panel, before/after view
- Use Recharts or D3 for visualization

Hour 13–15: Integration Testing

- End-to-end: UI → benchmark → stream traces → Gantt → metrics
- Test with 3-5 different tasks, fix timing/rendering issues

Hour 15–18: Demo Script + Polish

- Write exact demo script, add visual animations on metric improvements
- Pre-compute fallback results in case live demo has issues

Phase 4: Hardening (Hours 18–24)

- Run under concurrent load to demonstrate agentic headers impact
- Collect final benchmark numbers, generate comparison charts
- Build 3-5 presentation slides, record backup demo video
- Final bug fixes, ensure demo runs clean 3x in a row

Stretch Goals

- Full Gantt chart UI with real-time streaming
- Speculative decoding with measurable impact
- Agentic headers with priority scheduling under concurrent load
- NemoClaw sandbox with YAML security policies demonstrated

4. Exact Setup Instructions

Prerequisites

- Linux with NVIDIA GPU (A100 / H100 / RTX 4090 recommended)
- CUDA 12.x, Python 3.10+
- 40GB+ GPU memory for 30B models, or 16GB for 8B models

Step 1: Install vLLM

```
python -m venv vllm-env
source vllm-env/bin/activate
VLLM_USE_PRECOMPILED=1 pip install vllm
```

Step 2: Choose Your Model

Option A — Nemotron-Nano 30B (RECOMMENDED): Hybrid Mamba-Transformer MoE, only 3B active params out of 30B total. Best agentic reasoning accuracy per FLOP. Native tool calling. FP8 and BF16 variants.

Option B — Qwen3-8B: Strong tool calling, fits on single 24GB GPU. Good fallback.

Step 3: Serve with vLLM

```
export VLLM_ATTENTION_BACKEND=FLASHINFER

vllm serve nvidia/NVIDIA-Nemotron-Nano-3-30B-A3B-BF16 \
--dtype auto --trust-remote-code \
--served-model-name nemotron \
--host 0.0.0.0 --port 5000 \
--enable-auto-tool-choice \
--tool-call-parser qwen3_coder \
--reasoning-parser deepseek_r1 \
--enable-prefix-caching \
--max-model-len 32768
```

For speculative decoding, add:

```
--speculative-model [DRAFT_MODEL_PATH] \
--num-speculative-tokens 5
```

For smaller GPUs (Qwen3-8B):

```
vllm serve Qwen/Qwen3-8B \
--dtype auto --trust-remote-code \
--served-model-name agent-model \
```

```
--host 0.0.0.0 --port 5000 \  
--enable-auto-tool-choice \  
--tool-call-parser hermes \  
--enable-prefix-caching \  
--max-model-len 16384
```

Step 4: Install NemoClaw

```
curl -fsSL https://raw.githubusercontent.com/NVIDIA/NemoClaw/main/install.sh | bash  
nemoclaw my-agent onboard
```

Step 5: Install NeMo Agent Toolkit

```
git clone https://github.com/NVIDIA/NeMo-Agent-Toolkit.git  
cd NeMo-Agent-Toolkit && pip install -e .
```

Step 6: Verify — Test Tool Calling

```
import openai  
  
client = openai.OpenAI(base_url="http://localhost:5000/v1", api_key="dummy")  
  
response = client.chat.completions.create(  
    model="nemotron",  
    messages=[{"role": "user", "content": "Search for AI news"}],  
    tools=[{"type": "function", "function": {  
        "name": "search_web",  
        "description": "Search the web",  
        "parameters": {"type": "object", "properties": {  
            "query": {"type": "string"}, "required": ["query"]}}}],  
    tool_choice="auto", max_tokens=500)  
print(response.choices[0].message.tool_calls)
```

5. Optimization Tricks (Critical)

A. Prefix Caching — The Single Biggest Win

Agent loops repeat the same system prompt + tool definitions on every LLM call. Without prefix caching, vLLM recomputes KV states for all of this every time. A typical agentic prompt has 2K-4K tokens of prefix. At 100 tok/ms prefill speed, that's 20-40ms per call. Over 4 calls in a loop, you save 60-120ms. Under longer contexts (8K+ accumulated conversation), savings grow to 200-800ms per call.

Key trick: Structure prompts so the common prefix is maximally long. Put ALL tool definitions in the system message (not per-turn). Put static instructions first, dynamic content last. This ensures maximum KV cache hit rates across calls.

B. Async Parallel Tool Execution

Don't execute tools sequentially when they're independent. Build a dependency graph from the agent's plan, group independent tools, fire them concurrently with `asyncio.gather()`. vLLM's continuous batching means concurrent LLM requests share GPU compute automatically.

C. Prompt Design for Speed

- **Constrained max_tokens:** Tool calls are ~50-200 tokens. Don't set `max_tokens=4096`.
- **JSON response format:** Structured output helps speculative decoding (predictable tokens).
- **Few-shot tool call examples in system prompt:** Improves accuracy AND spec decode acceptance rate.
- **Thinking budget control:** Nemotron supports it — "Respond immediately with function call, do not reason extensively."

D. Speculative Decoding for Tool Calls

Biggest wins when: output is structured JSON (high acceptance rate), batch sizes are small (1-4), and GPU has spare capacity. Use EAGLE-3 drafters from RedHatAI's HuggingFace collection. Helps most on the final answer generation (longest output). 1.5-1.7x speedup typical.

E. FP8 KV Cache

Add `--kv-cache-dtype fp8` to halve KV cache memory. Doubles the context length you can serve, gives more room for long agent conversations without quality loss.

F. Agentic Headers for Priority Scheduling

Under concurrent load, `nvext` headers tell vLLM which requests matter most. High priority for first call (time-to-first-activity) and last call (time-to-final-answer). Low priority for intermediate calls running parallel with longer siblings.

Meetup result: Agentic headers alone gave **-62% TTFT at medium utilization** and **-89% TTFT at high utilization**. Thompson sampling routing reached **3443 tok/s** (6.4x over round-robin's 535 tok/s).

G. Combined Optimization Checklist

Flag / Config	vLLM Command	When to Use
Prefix caching	--enable-prefix-caching	ALWAYS (free win for agents)
FP8 KV cache	--kv-cache-dtype fp8	When context is long / memory tight
Spec decoding	--speculative-model [PATH] --num-speculative-tokens 5	When batch size is small (1-4)
JSON mode	response_format={"type": "json_object"}	For structured tool call outputs
Tool call parser	--enable-auto-tool-choice --tool-call-parser qwen3_coder	ALWAYS for tool-calling agents
Low max_tokens	max_tokens=200 in API call	For short tool call outputs

6. Demo Strategy

The 2-Minute Script

[0:00–0:15] Hook

"Every AI agent is bottlenecked by inference. We built a system that profiles agent workflows, identifies bottlenecks in real-time, and auto-optimizes them using vLLM and NeMo Agent Toolkit. Watch."

[0:15–0:45] Baseline Run

- Click a task: "Compare the economic policies of three countries"
- Gantt chart populates live: 4 LLM calls (red bars), 6 tool calls (blue bars)
- Point out: "Each LLM call re-processes the full 3K-token prefix. Tools A and B could run in parallel but don't."
- **Metrics:** "Total: 8.2 seconds. Average TTFT: 890ms."

[0:45–1:00] Optimizer Agent Analyzes

- Recommendations appear: "Enable prefix caching: saves ~1.0s"
- "Parallelize web_search + db_query: saves ~1.2s"
- "Speculative decoding: 1.6x decode speedup on final answer"

[1:00–1:30] Optimized Run

- Toggle all optimizations ON. Same task. Gantt bars visibly shorter. Tools overlap.
- **Metrics flash:** "Total: 3.4s. TTFT: 120ms. **2.4x faster.**"

[1:30–2:00] The Punchline

- Side-by-side Gantt charts. Before/after metrics.
- "Prefix caching alone: 7.4x TTFT improvement. Combined: 2.4x end-to-end."
- "All running on a single A100. No cloud APIs. Fully local with Nemotron."

Metrics to Highlight

Metric	Baseline	Optimized	Improvement
End-to-end latency	~8.2s	~3.4s	2.4x faster
Average TTFT / call	~890ms	~120ms	7.4x faster
Decode throughput	~45 tok/s	~72 tok/s	1.6x faster
GPU utilization	~40%	~78%	~2x better
Prefix cache hit rate	0%	87%	N/A

How to Make It Look 'Insane'

- **Real-time Gantt chart:** Judges have never seen agent execution visualized this way. Bars shrinking is visceral.
- **Live metrics counter:** TTFT ticking down from 890ms to 120ms with dramatic animation.
- **Concurrent load test:** 10 agents running simultaneously. Baseline: 30s. With agentic headers: 5s.

- **The "meta" angle:** The Optimizer Agent itself runs on vLLM with prefix caching. It analyzes its own traces. Inception.
- **Hardware badge:** "Single A100. Zero cloud APIs. 100% local inference with Nemotron."

Quick Reference Links

Resource	URL
vLLM	github.com/vllm-project/vllm
NeMo Agent Toolkit	github.com/NVIDIA/NeMo-Agent-Toolkit
NemoClaw	github.com/NVIDIA/NemoClaw
Nemotron-Nano on vLLM	blog.vllm.ai/2025/12/15/run-nvidia-nemotron-3-nano.html
Speculators (draft models)	github.com/vllm-project/speculators
LLM Compressor (quantization)	github.com/vllm-project/llm-compressor
AI-Q Blueprint	build.nvidia.com/nvidia/aiq
NAT Docs	docs.nvidia.com/nemo/agent-toolkit/latest/
RedHatAI Speculator Models	huggingface.co/collections/RedHatAI/speculator-models
vLLM Slack	slack.vllm.ai

Remember: The MVP (Subject Agent + prefix caching ON/OFF comparison + terminal metrics) must work by Hour 10. Everything else — Gantt UI, Optimizer Agent, spec decoding, agentic headers — is polish that makes the demo look insane. Prioritize ruthlessly. Ship the comparison first.